

Nome: Guilherme de Moraes China Costa

Data: 27/09/2026

Título: Projeto de Bloco: Engenharia de Softwares Escaláveis - TP5

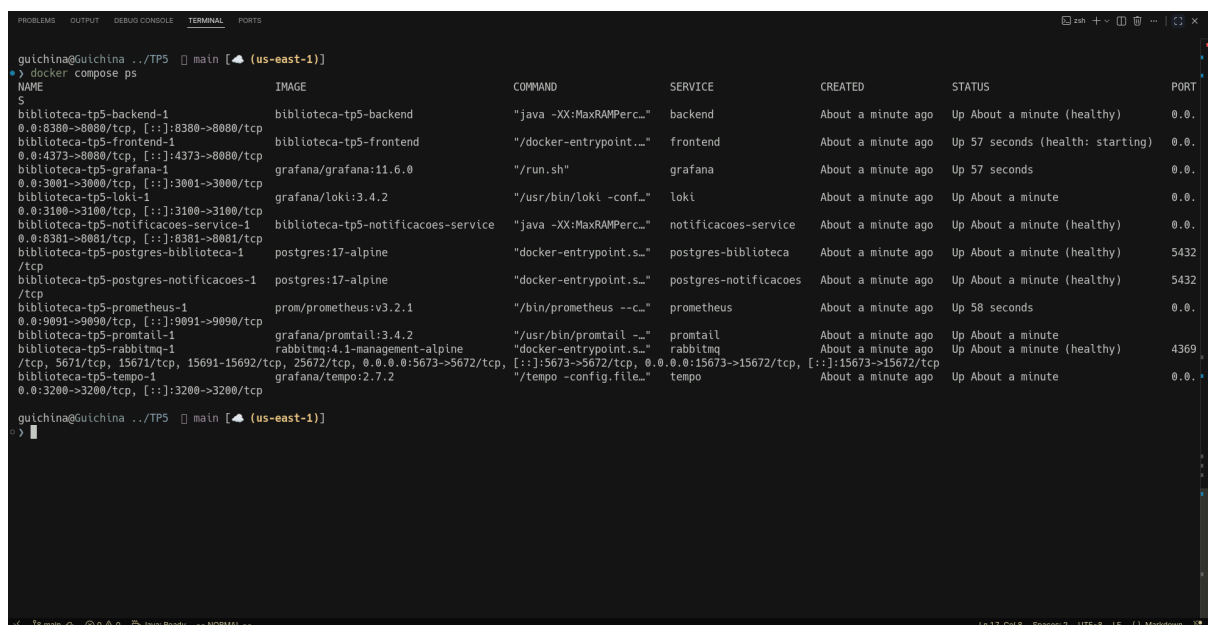
Preparação do Sistema para Operação

Link da implementação:

<https://github.com/guilhermecosta/biblioteca-tp5>

O sistema de biblioteca foi preparado para operação com Docker, Kubernetes, monitoramento e testes automatizados. A aplicação é formada pelo frontend React, pelo backend principal e pelo microserviço de notificações, que se comunicam por HTTP e eventos publicados no RabbitMQ.

O ambiente de execução também inclui dois bancos PostgreSQL independentes e uma estrutura de observabilidade composta por Prometheus, Loki, Promtail, Tempo e Grafana.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORT
biblioteca-tp5-backend-1	biblioteca-tp5-backend	"java -XX:MaxRAMPerC..."	backend	About a minute ago	Up About a minute (healthy)	0.0.
0.0:8380->8080/tcp, [::]:8380->8080/tcp						
biblioteca-tp5-frontend-1	biblioteca-tp5-frontend	"/docker-entrypoint..."	frontend	About a minute ago	Up 57 seconds (health: starting)	0.0.
0.0:4373->8080/tcp, [::]:4373->8080/tcp						
biblioteca-tp5-grafana-1	grafana/grafana:11.6.0	"/run.sh"	grafana	About a minute ago	Up 57 seconds	0.0.
0.0:3001->3000/tcp, [::]:3001->3000/tcp						
biblioteca-tp5-loki-1	grafana/loki:3.4.2	"/usr/bin/loki -conf..."	loki	About a minute ago	Up About a minute	0.0.
0.0:3100->3100/tcp, [::]:3100->3100/tcp						
biblioteca-tp5-notificacoes-service-1	biblioteca-tp5-notificacoes-service	"java -XX:MaxRAMPerC..."	notificacoes-service	About a minute ago	Up About a minute (healthy)	0.0.
0.0:8381->8081/tcp, [::]:8381->8081/tcp						
biblioteca-tp5-postgres-biblioteca-1	postgres:17-alpine	"docker-entrypoint.s..."	postgres-biblioteca	About a minute ago	Up About a minute (healthy)	5432
/tcp						
biblioteca-tp5-postgres-notificacoes-1	postgres:17-alpine	"docker-entrypoint.s..."	postgres-notificacoes	About a minute ago	Up About a minute (healthy)	5432
/tcp						
biblioteca-tp5-prometheus-1	prom/prometheus:v3.2.1	"/bin/prometheus --c..."	prometheus	About a minute ago	Up 58 seconds	0.0.
0.0:9091->9090/tcp, [::]:9091->9090/tcp						
biblioteca-tp5-promtail-1	grafana/promtail:3.4.2	"/usr/bin/promtail -..."	promtail	About a minute ago	Up About a minute	0.0.
0.0:9091->9090/tcp, [::]:9091->9090/tcp						
biblioteca-tp5-rabbitmq-1	rabbitmq:4.1-management-alpine	"docker-entrypoint.s..."	rabbitmq	About a minute ago	Up About a minute (healthy)	4369
/tcp, 5671/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:15673->15672/tcp						
biblioteca-tp5-tempo-1	grafana/tempo:2.7.2	"/tempo -config.file..."	tempo	About a minute ago	Up About a minute	0.0.
0.0:3200->3200/tcp, [::]:3200->3200/tcp						

Figura 1 - Serviços da aplicação e de observabilidade executados pelo Docker Compose.

A captura mostra os contêineres do backend, frontend, notificações, bancos e RabbitMQ saudáveis, além dos serviços usados para métricas, logs e rastreamento.

Arquitetura de operação

```
Usuário
|
Frontend React + Nginx
|
Backend Biblioteca ----- PostgreSQL Biblioteca
|
| eventos da Transactional Outbox
v
RabbitMQ ----- Serviço de Notificações ----- PostgreSQL Notificações

Prometheus ----- métricas do backend e notificações
Promtail + Loki --- agregação de logs
OpenTelemetry ----- traces enviados ao Tempo
Grafana ----- consulta de métricas, logs e traces
```

O Nginx serve o frontend e encaminha as requisições de **API** ao backend. Cada microserviço possui seu próprio banco. Empréstimos e devoluções geram eventos persistidos na Outbox e posteriormente publicados no RabbitMQ.

Contêineres

Os três componentes da aplicação possuem Dockerfiles com build em múltiplos estágios. As imagens finais contêm apenas os artefatos necessários, executam com usuários sem privilégios e possuem verificações de saúde.

Docker Compose

O Compose declara redes separadas para aplicação e observabilidade, volumes para os dados e dependências condicionadas aos health checks. As variáveis sensíveis podem ser definidas em arquivo **.env**, sem versionar valores locais.

Componentes executados

Grupo	Componentes
Aplicação	Frontend, backend e serviço de notificações.
Dados e eventos	PostgreSQL da Biblioteca, PostgreSQL de Notificações e RabbitMQ.
Observabilidade	Prometheus, Loki, Promtail, Tempo e Grafana.

O mesmo código é usado no Compose e no Kubernetes por meio do perfil de produção do Spring Boot, que troca o banco local H2 pelo PostgreSQL.

Implantação e escalabilidade com Kubernetes

Os manifestos são organizados com Kustomize. Os bancos e o RabbitMQ usam StatefulSets e armazenamento persistente. Backend, frontend e notificações usam Deployments e Services. ConfigMaps e Secrets separam as configurações das imagens.

As aplicações possuem probes de liveness e readiness, solicitações e limites de recursos. Backend e notificações iniciam com duas réplicas e possuem HorizontalPodAutoscaler entre duas e cinco réplicas, com alvo de 70% de CPU. PodDisruptionBudgets mantêm pelo menos uma réplica disponível durante interrupções voluntárias.

```
$ kubectl get pods -n biblioteca
NAME                                READY   STATUS    RESTARTS   AGE
backend-84546cb74f-dx25f            1/1     Running   0           30m
backend-84546cb74f-14649            1/1     Running   0           31m
frontend-fb4944c77-rjxdb            1/1     Running   3 (37m ago) 10h
frontend-fb4944c77-s6qxm            1/1     Running   3 (37m ago) 10h
grafana-5c99ddc65-h9bfg             1/1     Running   1 (10h ago) 10h
loki-5d66747b99-p5gzx              1/1     Running   1 (10h ago) 10h
notificacoes-service-5495bd94b5-jpbxq 1/1     Running   2 (37m ago) 10h
notificacoes-service-5495bd94b5-v6mvz 1/1     Running   2 (37m ago) 10h
postgres-biblioteca-0               1/1     Running   1 (10h ago) 10h
postgres-notificacoes-0             1/1     Running   1 (10h ago) 10h
prometheus-69b84b747f-97xj4         1/1     Running   1 (10h ago) 10h
promtail-n4jw9                      1/1     Running   0           22m
rabbitmq-0                           1/1     Running   2 (36m ago) 10h
tempo-57b4575585-djqpd              1/1     Running   2 (6m2s ago) 10h

$ kubectl get hpa -n biblioteca
NAME                                REFERENCE                                TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
backend                             Deployment/backend                       cpu: 34/70% 2         5         2           10h
notificacoes-service                Deployment/notificacoes-service          cpu: 14/70% 2         5         2           10h

$ kubectl get pdb -n biblioteca
NAME                                MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
backend                             1               N/A               1                      10h
notificacoes-service                1               N/A               1                      10h
```

Figura 2 - Pods, autoscalers e políticas de disponibilidade no namespace biblioteca.

Na execução apresentada, todos os pods estavam ativos. Os HPAs recebiam as métricas do Metrics Server e mantinham duas réplicas para os dois microsserviços.

```
kubectl apply -k k8s
kubectl get pods -n biblioteca
kubectl get hpa,pdb -n biblioteca
```

Monitoramento dos microsserviços

O Spring Boot Actuator expõe health checks e métricas no formato Prometheus. O Promtail executa como DaemonSet, descobre os pods do namespace e envia os logs ao Loki. Os serviços geram traces com Micrometer e OpenTelemetry e os exportam por OTLP para o Tempo.

O Grafana recebe os três datasources por provisionamento. Os logs possuem **traceld** e **spanId**, permitindo relacionar uma linha de log à transação distribuída correspondente.

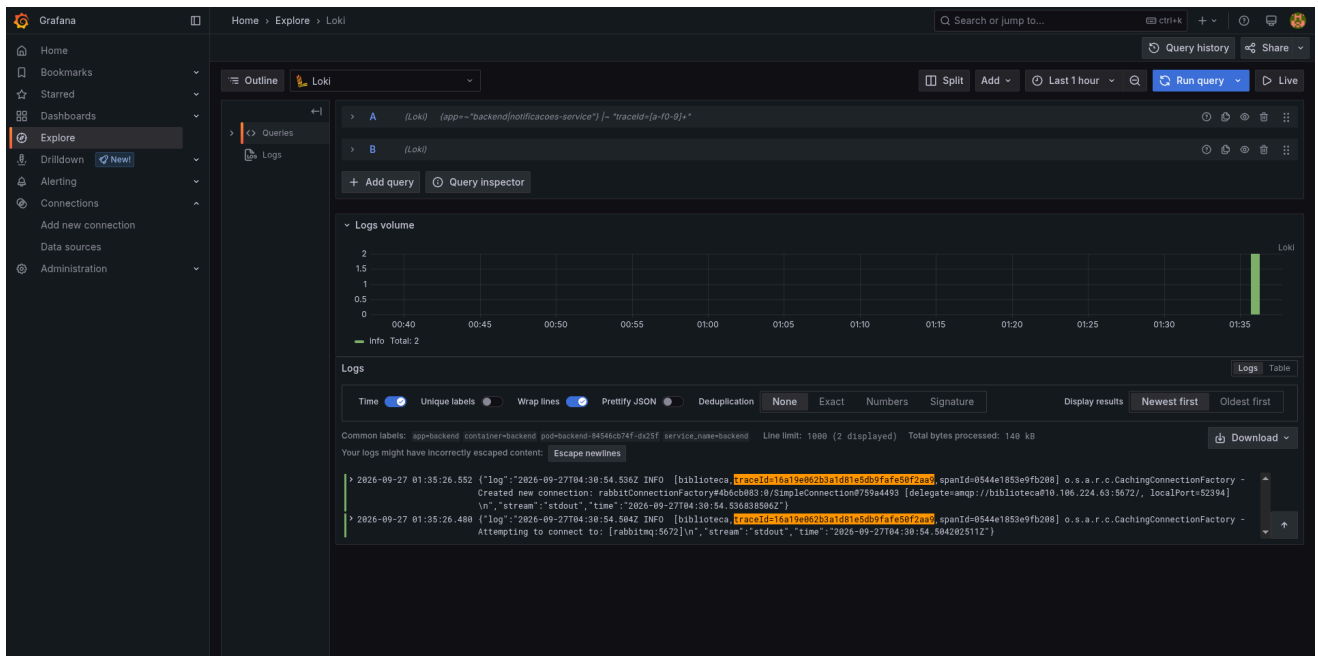
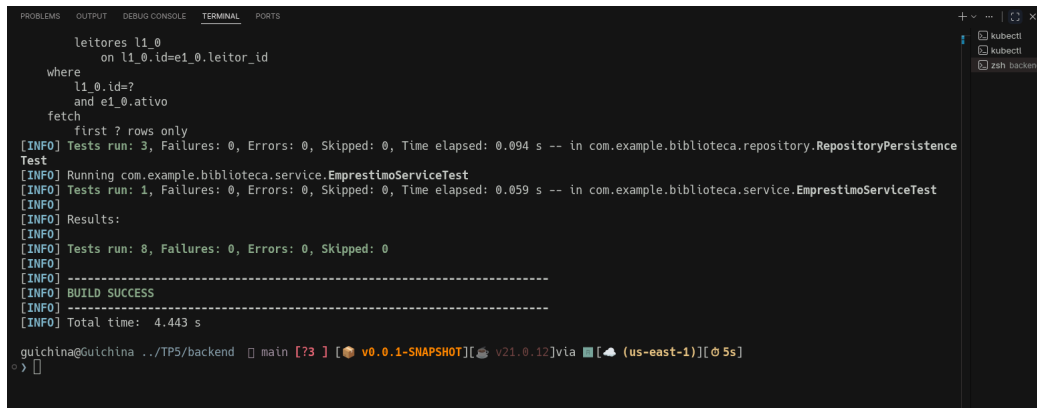


Figura 3 - Logs do backend consultados no Loki com traceld e spanId.

Ferramenta	Finalidade
Prometheus	Disponibilidade, requisições HTTP, JVM e consumo dos serviços.
Loki e Promtail	Coleta centralizada e pesquisa dos logs dos pods e contêineres.
Tempo	Armazenamento e consulta dos traces distribuídos.
Grafana	Consulta unificada e correlação entre métricas, logs e traces.

Testes dos componentes

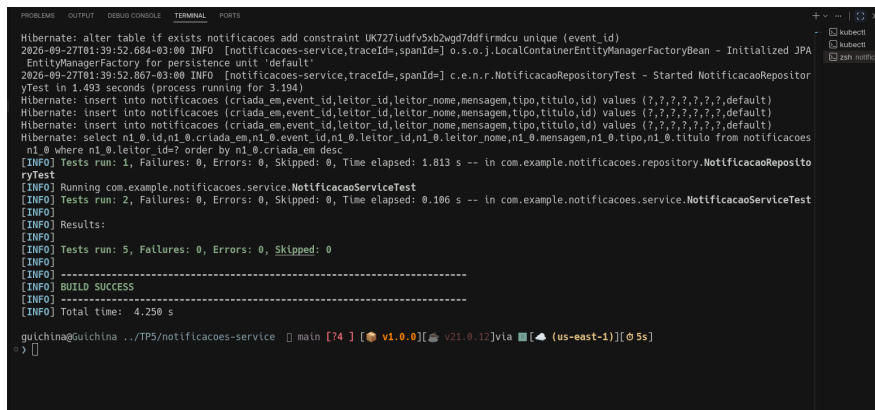
Os testes do backend cobrem regras de empréstimo e devolução, persistência, histórico das entidades, endpoints HTTP e registro dos eventos na Outbox.



```
leitores l1_0
on l1_0.id=e1_0.leitor_id
where
l1_0.id=?
and e1_0.ativo
fetch
first ? rows only
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.094 s -- in com.example.biblioteca.repository.RepositoryPersistenceTest
Test
[INFO] Running com.example.biblioteca.service.EmprestimoServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.059 s -- in com.example.biblioteca.service.EmprestimoServiceTest
[INFO] Results:
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 4.443 s
guichina@Guichina: ~/TP5/backend [73] [v0.0.1-SNAPSHOT] [v21.0.12] via [us-east-1] [05s]
```

Figura 4 - Oito testes do backend executados sem falhas.

O serviço de notificações possui testes de persistência, criação dos avisos, endpoint de consulta, consumo dos eventos e descarte de mensagens duplicadas pelo identificador do evento.



```
Hibernate: alter table if exists notificacoes add constraint UK727ludfy5xb2wgd7ddfrmdcu unique (event_id)
2026-09-27T01:39:52.684-03:00 INFO [notificacoes-service,traceId=,spanId=] o.s.o.j.LocalContainerEntityManagerFactoryBean - Initialized JPA
EntityManagerFactory for persistence unit 'default'
2026-09-27T01:39:52.867-03:00 INFO [notificacoes-service,traceId=,spanId=] c.e.n.r.NotificacaoRepositoryTest - Started NotificacaoRepositoryTest in 1.493 seconds (process running for 3.194)
Hibernate: insert into notificacoes (criada_em,event_id,leitor_id,leitor_nome,mensagem,tip,titulo,id) values (?,?,?,?,?,?,default)
Hibernate: insert into notificacoes (criada_em,event_id,leitor_id,leitor_nome,mensagem,tip,titulo,id) values (?,?,?,?,?,?,default)
Hibernate: insert into notificacoes (criada_em,event_id,leitor_id,leitor_nome,mensagem,tip,titulo,id) values (?,?,?,?,?,?,default)
Hibernate: select nl_0.id,nl_0.criada_em,nl_0.event_id,nl_0.leitor_id,nl_0.leitor_nome,nl_0.mensagem,nl_0.tipo,nl_0.titulo from notificacoes
nl_0 where nl_0.leitor_id=? order by nl_0.criada_em desc
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.813 s -- in com.example.notificacoes.repository.NotificacaoRepositoryTest
[INFO] Running com.example.notificacoes.service.NotificacaoServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in com.example.notificacoes.service.NotificacaoServiceTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 4.258 s
guichina@Guichina: ~/TP5/notificacoes-service [74] [v1.0.0] [v21.0.12] via [us-east-1] [05s]
```

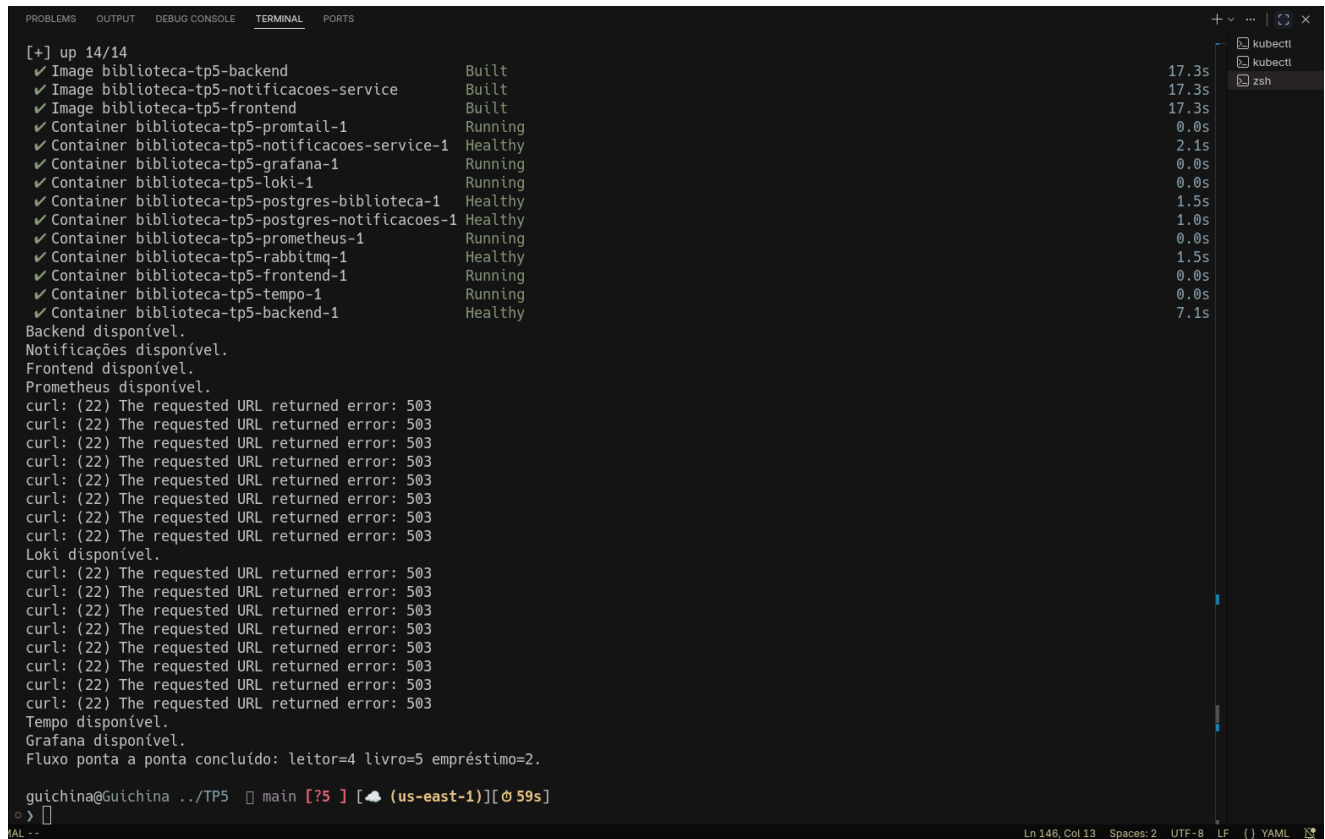
Figura 5 - Cinco testes do microserviço de notificações executados sem falhas.

Ao todo, os dois projetos Java executaram treze testes automatizados. O frontend também passa pelo comando de build na integração contínua.

Teste integrado

O script de teste monta a ponta constrói e inicia todo o ambiente Docker. Em seguida, verifica a disponibilidade dos componentes, cadastra um leitor e um livro, registra um empréstimo e aguarda a notificação gerada pelo evento.

Depois, o script registra a devolução, aguarda o segundo evento e confirma a exposição das métricas dos dois microsserviços. As respostas temporárias durante a inicialização são tratadas pelo mecanismo de espera até que cada serviço fique pronto.



```
[+] up 14/14
✓ Image biblioteca-tp5-backend Built 17.3s
✓ Image biblioteca-tp5-notificacoes-service Built 17.3s
✓ Image biblioteca-tp5-frontend Built 17.3s
✓ Container biblioteca-tp5-promtail-1 Running 0.0s
✓ Container biblioteca-tp5-notificacoes-service-1 Healthy 2.1s
✓ Container biblioteca-tp5-grafana-1 Running 0.0s
✓ Container biblioteca-tp5-loki-1 Running 0.0s
✓ Container biblioteca-tp5-postgres-biblioteca-1 Healthy 1.5s
✓ Container biblioteca-tp5-postgres-notificacoes-1 Healthy 1.0s
✓ Container biblioteca-tp5-prometheus-1 Running 0.0s
✓ Container biblioteca-tp5-rabbitmq-1 Healthy 1.5s
✓ Container biblioteca-tp5-frontend-1 Running 0.0s
✓ Container biblioteca-tp5-tempo-1 Running 0.0s
✓ Container biblioteca-tp5-backend-1 Healthy 7.1s
Backend disponível.
Notificações disponível.
Frontend disponível.
Prometheus disponível.
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
Loki disponível.
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
curl: (22) The requested URL returned error: 503
Tempo disponível.
Grafana disponível.
Fluxo ponta a ponta concluído: leitor=4 livro=5 empréstimo=2.

guichina@Guichina: ~/TP5 [?] [us-east-1] [0 59s]
```

Figura 6 - Cenário ponta a ponta concluído com empréstimo, devolução e notificações.

A mensagem final confirma que o cenário atravessou frontend, backend, PostgreSQL, RabbitMQ e o serviço de notificações. O mesmo fluxo também foi executado na implantação Kubernetes.

Integração e entrega contínuas

O workflow do GitHub Actions é iniciado em pushes e pull requests para as branches principal e de compatibilidade. Os testes do backend, notificações, frontend e manifestos Kubernetes são executados em paralelo antes do cenário integrado.

Após o teste ponta a ponta, a pipeline constrói e publica as imagens do backend, frontend e notificações no GitHub Container Registry. As imagens recebem uma tag baseada no commit e a branch principal também publica a versão **latest**.

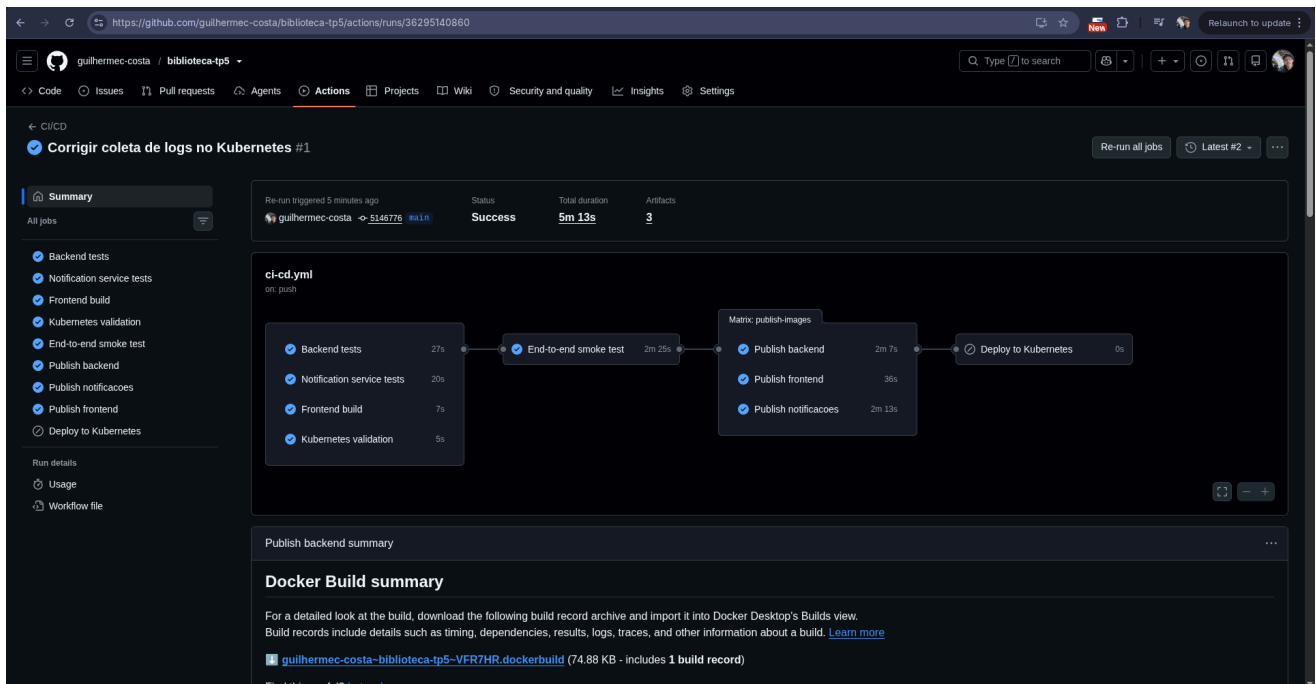


Figura 7 - Pipeline concluída com testes, validação e publicação das imagens.

O deploy em Kubernetes é opcional e permanece ignorado enquanto não existir um cluster externo configurado. Ao definir a variável **ENABLE_KUBERNETES_DEPLOY** e o secret **KUBE_CONFIG**, o job aplica os manifestos, atualiza as imagens e aguarda o rollout.

Gestão de configuração e conclusão

O projeto possui um repositório Git próprio e está publicado no GitHub. As mudanças de infraestrutura, aplicação, testes e documentação são mantidas no mesmo histórico, permitindo relacionar cada execução da pipeline ao commit que a originou.

Configuração operacional

- variáveis de ambiente para bancos, RabbitMQ, URLs internas e exportação OTLP;
- arquivo de exemplo para a configuração local, sem versionar o arquivo .env;
- ConfigMaps e Secrets no Kubernetes;
- health checks no Docker e probes no Kubernetes;
- limites de recursos, réplicas, HPA e PDB;
- imagens versionadas e publicadas pela pipeline.

Documentação entregue

O README descreve a arquitetura, os endereços locais, a execução com Docker Compose e Minikube, as consultas de monitoramento, os testes e a configuração do CI/CD. O roteiro de apresentação reúne as falas e a sequência da demonstração prática.

Conclusão

A aplicação passou a possuir um processo reproduzível de construção e implantação. Docker padroniza a execução dos componentes; Kubernetes fornece orquestração, recuperação e escala; Prometheus, Loki e Tempo tornam o ambiente observável; e o GitHub Actions automatiza testes, validação e publicação.

Os testes isolados e integrados verificam tanto as regras dos componentes quanto o funcionamento conjunto. Dessa forma, falhas podem ser detectadas antes da publicação, e a operação pode ser acompanhada por métricas, logs correlacionados e traces distribuídos.

Principais arquivos

Arquivo ou diretório	Conteúdo
docker-compose.yml	Ambiente local completo.
k8s/	Implantação, dados, observabilidade e escalabilidade.
monitoring/	Configurações do Prometheus, Loki, Promtail, Tempo e Grafana.
.github/workflows/ci-cd.yml	Pipeline de integração e entrega contínuas.
scripts/smoke-test.sh	Cenário automatizado ponta a ponta.